Experiments 1 - 5

Regular Expressions, Finite Automata, Morphological Analysis and Stemming (Python)

Experiment 1: Regular Expressions in Python

Aim: Write a program that demonstrates how to use regular expressions in Python to match and search for patterns in text.

Program

```
import re

text = ("Contact us at support@example.com or sales@company.org. "
        "Call 044-2345-6789 or 98765-43210. Meeting on 27/10/2026 at 10:30.")

# 1. re.search - first occurrence
m = re.search(r"\b\d{2}/\d{2}/\d{4}\b", text)
print("Date found      :", m.group() if m else None, "at", m.span())

# 2. re.findall - all e-mail addresses
emails = re.findall(r"[\w.+-]+@[\w-]+(?:\.[\w-]+)+", text)
print("E-mails         :", emails)

# 3. re.match - only matches at the beginning
print("Starts 'Contact':", bool(re.match(r"Contact", text)))
print("Starts 'sales'  :", bool(re.match(r"sales", text)))

# 4. phone numbers
phones = re.findall(r"\b\d{2,5}-\d{4,5}(?:-\d{4})?\b", text)
print("Phone numbers   :", phones)

# 5. re.sub - replace pattern
print("Masked text     :", re.sub(r"\d", "#", "Order 1234 costs 567"))

# 6. re.split
print("Split on , ; .  :", re.split(r"[,;.]\s*", "apple, banana; cherry. mango"))

# 7. groups
m = re.search(r"(\d{2}):(\d{2})", text)
print("Time (hh, mm)   :", m.groups())

# 8. finditer with positions
for mt in re.finditer(r"\b[A-Z][a-z]+\b", text):
    print("Capitalised word:", mt.group(), mt.span())
```

Output

```
Date found      : 27/10/2026 at (102, 112)
E-mails         : ['support@example.com', 'sales@company.org']
Starts 'Contact': True
Starts 'sales'  : False
Phone numbers   : ['044-2345-6789', '98765-43210']
Masked text     : Order #### costs ###
Split on , ; .  : ['apple', 'banana', 'cherry', 'mango']
Time (hh, mm)   : ('10', '30')
Capitalised word: Contact (0, 7)
Capitalised word: Call (56, 60)
Capitalised word: Meeting (91, 98)
```

Experiment 2: Finite State Automaton - Strings Ending with 'ab'

Aim: Implement a basic finite state automaton that recognizes strings ending with 'ab' using Python.

Program

```
# DFA that accepts strings over {a, b} ending with 'ab'
class DFA:
    def __init__(self):
        self.start = 'q0'
        self.accepting = {'q2'}
        # q0: no progress, q1: last char was 'a', q2: last two chars were 'ab'
        self.delta = {
            ('q0', 'a'): 'q1', ('q0', 'b'): 'q0',
            ('q1', 'a'): 'q1', ('q1', 'b'): 'q2',
            ('q2', 'a'): 'q1', ('q2', 'b'): 'q0',
        }

    def run(self, s):
        state = self.start
        path = [state]
        for ch in s:
            if (state, ch) not in self.delta:
                return False, path
            state = self.delta[(state, ch)]
            path.append(state)
        return state in self.accepting, path

dfa = DFA()
tests = ["ab", "aab", "bab", "abab", "aabb", "ba", "a", "b", "", "bbbab", "abba"]
print(f"{'String':<10}{'Result':<10}Path")
for t in tests:
    ok, path = dfa.run(t)
    print(f"{repr(t):<10}{'ACCEPT' if ok else 'REJECT':<10}{' -> '.join(path)}")
```

Output

```
String    Result    Path
'ab'      ACCEPT    q0 -> q1 -> q2
'aab'     ACCEPT    q0 -> q1 -> q1 -> q2
'bab'     ACCEPT    q0 -> q0 -> q1 -> q2
'abab'    ACCEPT    q0 -> q1 -> q2 -> q1 -> q2
'aabb'    REJECT    q0 -> q1 -> q1 -> q2 -> q0
'ba'      REJECT    q0 -> q0 -> q1
'a'       REJECT    q0 -> q1
'b'       REJECT    q0 -> q0
''        REJECT    q0
'bbbab'   ACCEPT    q0 -> q0 -> q0 -> q0 -> q1 -> q2
'abba'    REJECT    q0 -> q1 -> q2 -> q0 -> q1
```

Experiment 3: Morphological Analysis using NLTK

Aim: Write a program that demonstrates how to perform morphological analysis using the NLTK library in Python.

Program

```
from nltk.stem import PorterStemmer, LancasterStemmer, SnowballStemmer
from nltk.tokenize import wordpunct_tokenize

text = "The children were running quickly while the happier dogs jumped over fences."
tokens = [t.lower() for t in wordpunct_tokenize(text) if t.isalpha()]
print("Tokens:", tokens, "\n")

porter = PorterStemmer()
lancaster = LancasterStemmer()
snowball = SnowballStemmer("english")

print(f"{'Word':<12}{'Porter':<10}{'Lancaster':<12}{'Snowball':<10}Suffix removed (Porter)")
for w in tokens:
    s = porter.stem(w)
    suffix = w[len(s):] if w.startswith(s) else "-"
    print(f"{w:<12}{s:<10}{lancaster.stem(w):<12}{snowball.stem(w):<10}{suffix or '-'}")
```

Output

```
Tokens: ['the', 'children', 'were', 'running', 'quickly', 'while', 'the', 'happier', 'dogs', 'jumped', 'over', 'fences']

Word        Porter    Lancaster   Snowball  Suffix removed (Porter)
the         the       the         the       -
children    children  childr      children  -
were        were      wer         were      -
running     run       run         run       ning
quickly     quickli   quick       quick     -
while       while     whil        while     -
the         the       the         the       -
happier     happier   happy       happier   -
dogs        dog       dog         dog       s
jumped      jump      jump        jump      ed
over        over      ov          over      -
fences      fenc      fent        fenc      es
```

Experiment 4: Finite-State Machine for Morphological Parsing (Plural Nouns)

Aim: Implement a finite-state machine for morphological parsing that generates plural forms of English nouns using Python.

Program

```
# Finite-state machine for English noun pluralisation
# States: START -> (classify ending) -> ADD_S / ADD_ES / Y_TO_IES / F_TO_VES / DONE
IRREGULAR = {"child": "children", "man": "men", "woman": "women", "foot": "feet",
             "tooth": "teeth", "mouse": "mice", "person": "people", "sheep": "sheep"}
VOWELS = "aeiou"
O_EXCEPTIONS = {"photo", "piano", "halo", "radio", "video", "zoo"}  # nouns ending in -o that just add -s

def pluralize(noun):
    trace = ["START"]
    n = noun.lower()
    if n in IRREGULAR:
        trace.append("IRREGULAR")
        return IRREGULAR[n], trace + ["DONE"]
    if n.endswith(("s", "x", "z", "ch", "sh")):
        trace.append("ADD_ES"); result = n + "es"
    elif n.endswith("y") and len(n) > 1 and n[-2] not in VOWELS:
        trace.append("Y_TO_IES"); result = n[:-1] + "ies"
    elif n.endswith("fe"):
        trace.append("FE_TO_VES"); result = n[:-2] + "ves"
    elif n.endswith("f"):
        trace.append("F_TO_VES"); result = n[:-1] + "ves"
    elif n.endswith("o") and n not in O_EXCEPTIONS and len(n) > 1 and n[-2] not in VOWELS:
        trace.append("ADD_ES"); result = n + "es"
    else:
        trace.append("ADD_S"); result = n + "s"
    return result, trace + ["DONE"]

words = ["cat", "dog", "bus", "box", "buzz", "church", "dish", "city", "baby",
         "boy", "key", "knife", "wolf", "leaf", "hero", "photo", "piano", "child", "man", "mouse", "sheep"]
print(f"{'Singular':<10}{'Plural':<10}State path")
for w in words:
    p, t = pluralize(w)
    print(f"{w:<10}{p:<10}{' -> '.join(t)}")
```

Output

```
Singular  Plural    State path
cat       cats      START -> ADD_S -> DONE
dog       dogs      START -> ADD_S -> DONE
bus       buses     START -> ADD_ES -> DONE
box       boxes     START -> ADD_ES -> DONE
buzz      buzzes    START -> ADD_ES -> DONE
church    churches  START -> ADD_ES -> DONE
dish      dishes    START -> ADD_ES -> DONE
city      cities    START -> Y_TO_IES -> DONE
baby      babies    START -> Y_TO_IES -> DONE
boy       boys      START -> ADD_S -> DONE
key       keys      START -> ADD_S -> DONE
knife     knives    START -> FE_TO_VES -> DONE
wolf      wolves    START -> F_TO_VES -> DONE
leaf      leaves    START -> F_TO_VES -> DONE
hero      heroes    START -> ADD_ES -> DONE
photo     photos    START -> ADD_S -> DONE
piano     pianos    START -> ADD_S -> DONE
child     children  START -> IRREGULAR -> DONE
man       men       START -> IRREGULAR -> DONE
mouse     mice      START -> IRREGULAR -> DONE
sheep     sheep     START -> IRREGULAR -> DONE
```

Experiment 5: Word Stemming using the Porter Stemmer

Aim: Use the Porter Stemmer algorithm to perform word stemming on a list of words using Python libraries.

Program

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()
words = ["running", "runs", "runner", "easily", "fairly", "caresses", "ponies",
         "happiness", "connection", "connected", "connecting", "generalization",
         "studies", "studying", "agreed", "plastered", "motoring", "relational"]

print(f"{'Word':<18}{'Stem'}")
print("-" * 30)
for w in words:
    print(f"{w:<18}{stemmer.stem(w)}")

sentence = "The students are studying connected algorithms and running experiments"
print("\nSentence :", sentence)
print("Stemmed  :", " ".join(stemmer.stem(w) for w in sentence.split()))
```

Output

```
Word              Stem
------------------------------------
running           run
runs              run
runner            runner
easily            easili
fairly            fairli
caresses          caress
ponies            poni
happiness         happi
connection        connect
connected         connect
connecting        connect
generalization    gener
studies           studi
studying          studi
agreed            agre
plastered         plaster
motoring          motor
relational        relat

Sentence : The students are studying connected algorithms and running experiments
Stemmed  : the student are studi connect algorithm and run experi
```